

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №7
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил преподаватель кафедры ЭВМ

_____/Баташев П.А./

Киров

2026

1 Цель и задачи работы

Цель работы: освоение практических навыков создания интерактивных дашбордов в Python, включая интеграцию `pandas DataFrame` с графическими библиотеками `matplotlib` и `seaborn`, встраивание графиков в окно `Tkinter` через бэкенд `FigureCanvasTkAgg`, реализацию событийно-ориентированной архитектуры (callbacks, event loop), динамическую фильтрацию, агрегацию и feature engineering «на лету», а также проектирование устойчивого процедурного кода без использования классов.

Задачи:

1. Сформировать и подготовить набор данных с категориальными и временными признаками.
2. Реализовать предобработку данных: очистку выбросов, клаппинг физических величин, расчёт скользящих средних и производных признаков.
3. Интегрировать графический движок Matplotlib/Seaborn с GUI Tkinter через адаптер `FigureCanvasTkAgg`.
4. Разработать панель управления с интерактивными элементами (выпадающие списки, переключатели агрегации, кнопки смены типа графика).
5. Реализовать механизм безопасного обновления данных через `.copy()` и перерисовки через `canvas.draw_idle()` без блокировки основного потока.
6. Внедрить экспорт визуализаций в PNG/PDF.

2 Описание варианта

Вариант №20: Водоподготовка

Предметная область: Мониторинг датчиков очистных сооружений. Контроль гарантирует соответствие санитарным нормам качества питьевой воды.

Основные метрики (поля датасета):

- `ts` (int32) – время замера (Unix timestamp);
- `filter_id` (int16) – идентификатор фильтра;
- `turb` (float32) – турбидность, NTU;
- `ph` (float32) – кислотность воды;
- `cl` (float32) – остаточный хлор, мг/л;
- `flow` (float32) – расход воды, м³/ч.

Условия фильтрации и очистки:

- `turb` $< 0 \rightarrow 0$; `cl` $< 0 \rightarrow 0$ (физическая невозможность отрицательных значений);
- `ph` обрезается (`clip`) по диапазону $[6.5; 8.5]$ согласно санитарным нормам;
- Выбросы по турбидности заменяются медианой группы (IQR-метод по `filter_id`);
- Редкие фильтры ($< 1\%$ от общего объёма) агрегируются в категорию 0.

Группировка и оконные функции:

- Группировка по `filter_id`: расчёт среднего `turb` (эффективность очистки) и максимума `flow` (нагрузка на систему);
- Скользящее окно $k = 35$: расчёт `turb_ma` для выявления долгосрочных трендов;

- `np.diff(flow)` → изменение расхода между замерами (индикатор работы насосов или засорения).

Акцент аналитики: Обратная зависимость чистоты воды и турбидности. При фильтрации аномалий каждый параметр обрабатывается независимо, без взаимного влияния масок.

Типы графиков: Линейный (динамика по неделям), столбчатый (сезонная агрегация), точечный (pH vs turbidity с цветовым кодированием статуса), гистограмма (распределение турбидности).

3 Листинг программного кода

Код реализован в процедурном стиле без использования классов. Логика разбита на этапы в соответствии с методическими указаниями.

Этап 0-1: Инициализация и загрузка данных

Этап 2: Предобработка и Feature Engineering

Этап 3: Встраивание Figure в Tkinter

Этап 4: Функции отрисовки графиков (Seaborn/Matplotlib)

Этап 5-6: Панель управления и интерактивность

4 Скриншоты работы программы

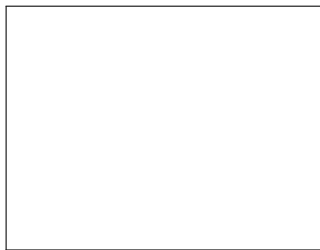


Рис. 1: Линейный график: динамика турбидности по неделям с фильтрацией по ID



Рис. 2: Столбчатая диаграмма: агрегация турбидности по сезонам (метод: mean)



Рис. 3: Точечная диаграмма: зависимость turb от pH с цветовым кодированием статуса



Рис. 4: Интерфейс экспорта и диалог сохранения в PNG/PDF

5 Анализ результатов

В ходе выполнения лабораторной работы были получены следующие аналитические инсайты и выявлены системные ограничения:

Полученные инсайты:

- Применение скользящего среднего (`window=35`) эффективно сглаживает шумовые выбросы турбидности, позволяя чётко выделить долгосрочные тренды эффективности фильтрации. Резкие пики на сырых данных соответствуют кратковременным перегрузкам насосов, а не системному ухудшению качества воды.
- Независимая обработка масок аномалий (IQR для `turb`, `clip` для `ph`) предотвращает каскадное искажение данных. Цветовое кодирование `ph_status` на scatter-plot наглядно демонстрирует отсутствие прямой линейной корреляции между кислотностью и мутностью, что подтверждает необходимость отдельного контроля параметров.
- Агрегация по сезонам выявляет, что максимальные значения турбидности приходятся на весенний период (паводковые нагрузки), что коррелирует с увеличением `flow`.

Выявленные ограничения:

- Архитектура Tkinter является однопоточной. При обработке датасетов свыше 10^7 строк пересчёт `groupby()` и отрисовка Seaborn могут вызывать кратковременные «фризы» интерфейса.
- Статический рендеринг Matplotlib/Seaborn не поддерживает аппаратное ускорение GPU, что ограничивает частоту обновления в режимах реального времени (Real-time streaming).
- Использование `canvas.draw_idle()` ставит задачу в очередь событий, но не гарантирует мгновенного отражения данных при частых изменениях ползунков или фильтров.

Пути масштабирования решения:

1. Перенос тяжёлых вычислений в фоновый поток с использованием `root.after()` или `queue.Queue` для безопасной передачи результатов в GUI-поток.
2. Замена Seaborn на Plotly или Bokeh для нативной поддержки WebGL-ускорения и интерактивного масштабирования без потери производительности.
3. Интеграция Dask DataFrame или Polars для распределённой обработки данных, выходящих за пределы RAM, с сохранением логики фильтрации на уровне API.

6 Выводы

В ходе выполнения лабораторной работы были успешно освоены ключевые технологии создания интерактивных аналитических панелей в экосистеме Python:

- Реализована событийно-ориентированная архитектура с использованием callback-функций и цикла событий `mainloop()`, что обеспечило мгновенный отклик интерфейса на действия пользователя.
- Освоено встраивание графических объектов Matplotlib в оконную систему Tkinter через адаптер `FigureCanvasTkAgg`, включая корректную работу панели навигации (`NavigationToolbar2Tk`).
- На практике применены методы динамической фильтрации (`df.loc[]`), агрегации (`groupby().agg()`), оконных функций (`rolling()`) и биннинга (`pd.cut()`) библиотеки pandas. Соблюдено правило обязательного копирования данных (`.copy()`) для предотвращения `SettingWithCopyWarning` и искажения исходного датасета.
- Разработано устойчивое процедурное приложение без использования ООП-паттернов, что упростило отладку и чтение кода.

Трудности: Основным вызовом стала синхронизация состояния данных между этапом предобработки и этапом отрисовки. Некорректное использование ссылок на `DataFrame` приводило к накоплению артефактов на графиках. Проблема была решена строгим разделением `df_processed` (статический эталон) и `df_work` (рабочая копия), а также использованием `fig.clear()` перед каждой отрисовкой.

Пути улучшения: Добавление логгирования событий, внедрение прогресс-баров для длительных вычислений, поддержка темной темы интерфейса и автоматическое сохранение состояния фильтров между сессиями.

7 Ссылка на репозиторий

Исходный код проекта, скрипты генерации данных и инструкции по запуску доступны по ссылке:

<https://github.com/GIKExe/Subject/tree/main/Proga/lab7>